

DOLLAR VIBE CLUB BUILD SPRINT

ShipGotchi Planning Pack

Paste your GitHub. Meet the creature your commits created. A focused 24-hour plan for a premium, no-login builder pet webapp designed to win the demo and feel launchable after the sprint.

Track: Vibe Gotchi

MVP: 5 Killer Features

No Login

Mobile-First

Cute Premium Toy

ShipGotchi Planning Index

Date: 2026-05-22 Challenge: Dollar Vibe Club 24-Hour Open Build Sprint Track: Vibe Gotchi Builder: Solo, full-stack Host: Vercel

Build Decisions (locked)

- **Solo execution.** No multi-person parallelism. The five workstreams in the execution plan run sequentially on a single critical path, optionally orchestrated as AI subagents.
- **Vercel hosting** with a serverless GitHub proxy. The proxy is **required, not optional** (see Technical Plan) — it is the single biggest defense against a dead live demo from rate limits.
- **Layered sprite system** (one base mascot + state-driven layers) is the primary art approach. Independently AI-generated variants are a stretch goal, not a dependency.
- **Build folder:** `C:\Code\Challenge` .

Strategic Decision

Build **ShipGotchi**: a no-login, mobile-first webapp where a visitor enters any public GitHub username and instantly hatches a premium digital pet whose mood, evolution, outfit, and Builder DNA are based on recent public GitHub activity.

The goal is to win the judging demo and make the build feel like something DVC would want to co-build after the sprint.

Planning Pack

1. [Product Brief](#)
2. [MVP Specification](#)
3. [Experience And Visual Direction](#)
4. [Technical Plan](#)
5. [Execution Plan And Agent Workstreams](#)
6. [Judging Demo And Submission Kit](#)
7. [Prompt Runbook](#)
8. [Sprite Asset Prompts](#)

Winning Angle

Most entries will either be over-scoped or too generic. ShipGotchi should win by being:

- Instant: no login, no setup, no tutorial.
- Emotional: the pet is the product, not a dashboard decoration.
- Streamable: judges can paste a username and get a fun reveal in seconds.
- Robust: GitHub public APIs, sample users, and graceful fallbacks keep the demo alive.
- Launchable: the long-term vision is a persistent builder companion that grows as you ship.

MVP Mantra

Paste your GitHub. Meet the creature your commits created.

ShipGotchi Product Brief

One-Liner

ShipGotchi is a premium digital toy for builders: paste a GitHub username, hatch a coding companion, and discover its mood, evolution, outfit, and Builder DNA based on how you ship.

Challenge Fit

The challenge asks for a consumer-facing webapp with at least three working features, no login wall, mobile-friendly execution, and a production URL that judges can test live.

ShipGotchi fits because:

- It is consumer-facing and immediately understandable.
- It uses public GitHub usernames, so there is no login or OAuth risk.
- It naturally supports several interactive features without feeling bloated.
- It is highly visual, which matters for a live judging stream.
- It aligns with Dollar Vibe Club's builder audience and post-sprint launch prize.

Audience

Primary users:

- Builders, vibe coders, indie hackers, students, and GitHub users.
- DVC stream viewers who want a quick, funny, shareable result.

Judges:

- Need to understand and test the product in under 60 seconds.
- Need to see at least three working features without reading instructions.
- Need a stable mobile-friendly production URL.

Product Promise

ShipGotchi turns dry public GitHub activity into a living, charming identity moment. It does not ask users to understand analytics. It gives them a creature.

MVP Scope

The MVP should contain exactly five standout features:

1. GitHub Hatch
2. Pet Mood And Evolution
3. Premium Sprite Moment
4. Builder DNA Card
5. Compare / Party Mode

These features are enough to feel complete, replayable, and demo-worthy without requiring accounts, payments, native apps, extensions, or complex infrastructure.

What We Are Not Building In The Sprint

- User accounts
- Global leaderboard
- Real desktop overlay pet
- Browser extension
- Native desktop app
- Real-time AI image generation
- GitHub OAuth
- Game economy
- Pet inventory
- Multiplayer persistence

Long-Term Vision

After the sprint, ShipGotchi can become a persistent builder companion:

- Lives as a web widget, browser extension, desktop app, or IDE sidebar.
- Grows as you ship.
- Celebrates commits, PRs, launches, and streaks.
- Adds friend leaderboards, teams, and seasonal collectible forms.

For the sprint, we only hint at this vision through a web-based "Desk Mode" where the pet can dock in the corner of the page.

ShipGotchi MVP Specification

MVP Goal

Deliver a polished, mobile-first webapp that produces a delightful result from any public GitHub username within 10 seconds.

Core Flow

1. User lands on ShipGotchi.
2. User enters a GitHub username or taps a sample username.
3. App fetches public GitHub profile, repos, and events.
4. Loading ritual plays: "Scanning commit aura..."
5. Pet hatches with animation.
6. User sees pet, mood, level, evolution, outfit, and Builder DNA.
7. User can add more usernames to compare pets locally.
8. User can generate a polished share card.

Feature 1: GitHub Hatch

User Story

As a visitor, I want to paste any public GitHub username so I can instantly hatch a pet without logging in.

Requirements

- Single prominent input on first screen.
- Sample usernames available as quick chips.
- Fetch public GitHub data:
 - User profile
 - Recent public events
 - Public repositories
- Fetch via our own `/api/github` proxy (authenticated token + caching), never the GitHub API directly. See Technical Plan.
- Show a premium loading state while data loads.
- Handle invalid usernames gracefully.
- Handle GitHub rate-limit or network failure with sample/demo data fallback (backstop only — the proxy is the primary defense).

Acceptance Criteria

- A judge can type a username and get a result without account creation.
- Empty, invalid, and rate-limited states do not break the UI.
- A sample username can produce a successful demo even if GitHub is unavailable.

Feature 2: Pet Mood And Evolution

User Story

As a visitor, I want the pet to visibly reflect the user's shipping activity so the result feels personal.

Inputs

- Recent push events
- Recent repository updates
- Top languages
- Account age
- Public repo count
- Recent event recency

Simple Scoring Model

Shipping Energy:

- High: multiple push/repo events in the last 7 days
- Medium: some activity in the last 30 days
- Low: no recent activity

Evolution Level:

- Egg: sparse or new account
- Sprout: low-to-medium activity
- Builder: consistent recent activity
- Overclocked: high recent activity
- Sleepy: quiet account

Mood:

- Hyped: active in last 24 hours
- Focused: active this week
- Curious: diverse languages/repos
- Sleepy: inactive for 30+ days
- Ancient Sage: old account with deep repo history

Acceptance Criteria

- Pet state changes for different usernames.
- Mood and evolution are explained in plain language.
- The scoring system is deterministic and fast.

Feature 3: Premium Sprite Moment

User Story

As a visitor, I want the pet reveal to feel premium and emotional.

Requirements

- One mascot species, rendered as **one base + state-driven layers** (body tint, expression, accessory, motion) — not 7 independently generated images. See Visual Direction "Sprite Strategy."
- Variants map to evolution/mood states.
- Art direction: cute premium toy, soft digital collectible, light builder-lab UI accents.
- Avoid requiring runtime AI image generation.
- Prefer SVG/CSS sprite components for guaranteed character consistency; pre-generated AI renders are an optional swap-in once MVP is stable.

Recommended Sprite Set

- Egg
- Sprout
- Builder
- Overclocked
- Sleepy
- Error/Mystery fallback
- Optional: language outfit overlays

Acceptance Criteria

- Pet is the visual centerpiece.
- Variants are visibly different.
- The result still works if images load slowly or fail.

Feature 4: Builder DNA Card

User Story

As a visitor, I want a stylish identity card that summarizes my builder personality.

Card Contents

- GitHub avatar and username
- ShipGotchi pet
- Builder class
- Level
- Mood
- Evolution stage
- Top language
- Estimated streak/activity signal
- One memorable personality line

Builder Class Examples

- Midnight Shipper
- Stack Alchemist
- Pixel Mechanic
- Patch Saint
- Launch Goblet
- Refactor Monk
- Repo Ranger
- Bug Whisperer
- Syntax Sprinter

Tone

Playful, affirming, slightly witty, never mean. The best copy feels like a fortune cookie for developers.

Acceptance Criteria

- The card is readable on mobile.
- The card makes the user want to screenshot/share.
- The card lists enough details that judges can see the GitHub data was used.

Feature 5: Compare / Party Mode

User Story

As a visitor, I want to compare multiple GitHub users so the product becomes replayable and stream-friendly.

Requirements

- Add up to four usernames in one session.
- Show pets side-by-side.
- Highlight differences:
 - Highest energy
 - Sleepiest pet
 - Most languages
 - Most ancient account
- Store only in browser/session state.

Acceptance Criteria

- Judges can compare sample users quickly.
- No backend persistence is required.
- Mobile layout remains usable.

Optional Nice-To-Have

Desk Mode:

- Pet docks in the bottom-right corner of the web page.
- Pet can be dragged within the page.
- This hints at the future desktop companion without violating the webapp-only rule.

Desk Mode should be added only after the five MVP features are stable.

ShipGotchi Experience And Visual Direction

Creative Direction

Base style: **cute premium toy**.

Supporting flavors:

- Light RPG evolution mechanics
- Soft builder-lab UI details
- Share-card polish

Avoid:

- Dark cyberpunk dominance
- Overly niche anime styling
- Dense dashboards
- Generic GitHub stats pages

Design North Star

The user should feel like they opened a premium collectible toy, not a developer analytics tool.

Visual Keywords

- Soft
- Tactile
- Collectible
- Rounded but not childish
- Bright
- Crisp
- Expressive
- Stream-friendly
- Screenshot-worthy

Mascot Direction

Use one mascot species with several forms.

This keeps execution manageable while preserving delight. Multiple species are a post-MVP expansion.

Mascot Personality

The mascot should feel:

- Loyal
- Slightly mischievous
- Proud when the user ships
- Sleepy when neglected
- Curious when the user explores many languages

Sprite Strategy (solo-safe: one base, layered states)

Primary approach — layered, not 7 separate renders. Build ONE base mascot, then drive its state with composable layers: body tint, eye/mouth expression, a small accessory, and a pose/animation tweak. This guarantees the variants read as *the same creature* (the hardest part of the "premium toy" promise) and avoids spending hours fighting an image model for character consistency.

- Body: a single SVG/CSS or sprite base.
- State layer: color + expression per evolution/mood.
- Accessory layer: optional outfit overlay by language.
- Motion layer: idle bob, hatch pop, sleepy z's — CSS or Framer Motion.

AI-generated full renders (runbook Prompt 6) are a **stretch goal** to swap in per variant if there's time after the five MVP features are stable — never a dependency for a working demo.

Sprite Variants (state targets for the layered system)

1. Egg: before enough data or first hatch
2. Sprout: early builder
3. Builder: stable activity
4. Overclocked: high activity
5. Sleepy: inactive
6. Mystery: invalid or hidden data

Outfit Overlay Ideas

- JavaScript/TypeScript: tiny star visor
- Python: soft wizard cap
- Rust: bronze armor accent
- Go: runner goggles
- CSS/HTML: paint scarf
- Java: coffee charm

Outfits should be decorative, not required for MVP correctness.

Page Structure

First Screen

Purpose: get to the hatch moment fast.

Elements:

- Brand name: ShipGotchi
- Literal promise: "Paste your GitHub. Meet the creature your commits created."
- Username input
- Hatch button
- Sample username chips
- Pet egg preview or tiny idle mascot

No long explanatory text.

Loading Ritual

Purpose: make waiting feel intentional.

Possible steps:

- Reading public commits
- Measuring shipping energy
- Sorting language snacks
- Polishing tiny goggles
- Hatching companion

Keep loading under a few seconds when possible.

Result Screen

Purpose: show the payoff immediately.

Primary visual:

- Large pet sprite
- Mood bubble
- Level/evolution badge

Secondary visual:

- Builder DNA card
- Activity meters
- Language trait
- Personality line

Actions:

- Compare another builder
- Generate/share card
- Try sample
- Desk Mode, if time allows

Copy Style

Voice:

- Warm
- Playful
- Premium
- Builder-native
- Not cringe

Examples:

- "Your ShipGotchi woke up caffeinated."
- "Recent commits detected. Tiny goggles equipped."
- "This pet has seen things in production."
- "Low activity, high potential. It is napping for strategic reasons."
- "Your top language shaped its outfit."

Accessibility And Usability

- High contrast text.
- Large touch targets.
- Keyboard-friendly username input.
- Mobile-first layout.
- No required hover interactions.
- Clear error states.
- Result should still make sense without animation.

ShipGotchi Technical Plan

Recommended Stack (locked: solo + Vercel)

- **Next.js + TypeScript** — chosen over Vite specifically so the GitHub proxy (below) lives in the same project as `app/api/*` routes, with zero extra deploy config on Vercel.
- Tailwind CSS for fast, consistent styling.
- All GitHub data fetched **through our own serverless proxy**, not directly from the browser.
- **Vercel** deployment (Git-connected, auto-deploy on push).

The sprint prioritizes speed, polish, and reliability over backend complexity. The one piece of backend we DO build — the proxy — exists purely to keep the live demo alive.

Data Sources

GitHub public REST API (called server-side from our proxy, never directly from the browser):

- `GET https://api.github.com/users/{username}`
- `GET https://api.github.com/users/{username}/events/public`
- `GET https://api.github.com/users/{username}/repos?sort=updated&per_page=20`

Optional:

- Repo language URLs returned by repo objects, if time allows.

GitHub Proxy (REQUIRED — build in Hour 0-2)

Why this is not optional: the unauthenticated GitHub REST API allows only **60 requests/hour per IP**. Our flow uses ~3 calls per username; a single 4-way Compare burns ~12. During live mobile judging, repeated tests — and multiple viewers behind the same network/IP — will exhaust 60 quickly, then every real fetch fails. A proxy with an authenticated token raises the ceiling to **5,000 requests/hour** and lets us cache.

Design:

- Single route: `GET /api/github?username={username}` .
- The route reads a `GITHUB_TOKEN` from Vercel environment variables (a fine-grained PAT with public read scope — no special permissions needed).
- The route makes the 3 GitHub calls server-side, normalizes them, and returns one combined payload (or a clean typed error).
- **Cache** responses for ~5 minutes (in-memory per warm lambda, or `Cache-Control` headers / a tiny edge cache). Repeated demo hits on the same username cost zero GitHub quota.
- The token is **never** exposed to the client. The browser only ever talks to `/api/github` .
- The route maps GitHub statuses to our UI states: `404` → `notFound` , `403/rate-limit` → `rateLimited` , `network/5xx` → `networkError` .

The bundled sample data (see scoring/error handling) remains the final safety net if the proxy itself is unavailable. Proxy = primary reliability; sample data = backstop.

Data Model

GitHubProfile

- login
- name
- avatarUrl
- publicRepos
- followers
- createdAt
- htmlUrl

GitHubActivity

- recentPushCount
- recentEventCount
- lastActivityAt
- repoUpdateCount
- topLanguages
- activeDaysEstimate
- accountAgeYears

PetProfile

- username
- displayName

- level
- mood
- evolutionStage
- builderClass
- topLanguage
- energyScore
- personalityLine
- spriteVariant
- outfitTrait

Scoring Logic

Keep scoring understandable and deterministic.

Energy Score

Start at 0.

- +30 if active in last 24 hours
- +20 if active in last 7 days
- +10 if active in last 30 days
- +5 per push event in recent public events, capped at 30
- +10 if repos updated recently
- +10 if public repo count is above 20
- +10 if more than 3 languages detected

Clamp score to 0-100.

Evolution Stage

- 0-15: Egg or Sleepy
- 16-35: Sprout
- 36-65: Builder
- 66-85: Evolved
- 86-100: Overclocked

If last activity is older than 30 days, prefer Sleepy unless account age/repo history is very strong.

Builder Class

Derive from:

- Top language
- Energy score
- Account age
- Repo diversity

Examples:

- High energy + TypeScript: Interface Sprinter
- High energy + Python: Script Sorcerer
- Old account + many repos: Ancient Maintainer
- Low recent activity + many repos: Sleeping Architect
- Diverse languages: Stack Alchemist

Error Handling

Invalid Username

Show Mystery pet and message:

"That GitHub user did not hatch. Check the spelling or try a sample builder."

Rate Limited

Show:

"GitHub is making us wait. Try a sample hatch while the tiny servers cool down."

Then provide sample data path.

Network Failure

Show retry and sample buttons.

Private/No Activity

Use profile/repo data if events are sparse. Do not make the user feel punished.

Performance

- Avoid runtime image generation.
- Preload key pet assets.
- Cache fetched users in memory/session storage.
- Keep first result under 10 seconds.
- Use skeleton/loading ritual instead of blank waiting.

Deployment Requirements

- **Vercel**, Git-connected, auto-deploy on push to main. Get a live URL in Hour 0-2 (deploy the empty scaffold first, then keep shipping to it).
- Set `GITHUB_TOKEN` in Vercel project env vars before the proxy goes live.
- Public production URL.
- No login required.
- Mobile-friendly.
- Stable until judging completes.
- Demo sample usernames built into UI.

ShipGotchi Execution Plan And Agent Workstreams

Build Strategy

We plan heavily, then execute. The product is small enough to complete in 24 hours, but polished enough to win if the workstreams stay focused.

Solo note: this is a single full-stack builder, so there is no human parallelism. The five workstreams below are a way to *organize* the work, not five people. Run them as one sequential critical path, or hand individual workstreams to AI subagents that you orchestrate. The timeline below is the sequential reality.

Recommended Timeline

Hour 0-2: Setup, Skeleton, And Live URL

- Scaffold Next.js + TypeScript + Tailwind app.
- **Deploy the empty scaffold to Vercel immediately** so a live URL exists from hour 1; every later change just ships to it.
- **Build the `/api/github` proxy** (authenticated token + ~5min cache) and set `GITHUB_TOKEN` in Vercel env. This is the reliability backbone — do it before the UI.
- Set up design tokens and responsive shell.
- Create the client fetch service that calls `/api/github` (not GitHub directly).
- Create sample data.
- Establish pet scoring types.

Hour 2-6: Core Loop

- Username input.
- Sample users.
- GitHub fetch.
- Scoring logic.
- Result page.
- Error and fallback states.

Hour 6-10: Visual Payoff

- Pet sprites/variants.
- Hatch animation.
- Builder DNA card.
- Premium mobile layout.
- Copywriting pass.

Hour 10-14: Compare Mode

- Add up to four users.
- Local session state.
- Side-by-side pet cards.
- Comparison highlights.

Hour 14-18: Polish And Demo

- Share card UI.
- Desk Mode if core features are stable.
- Mobile QA.
- Loading ritual.
- Empty/error states.

Hour 18-21: Hardening

- Test GitHub API failures.
- Test invalid usernames.
- Test mobile layout.
- Test production deployment.
- Verify no login required.

Hour 21-24: Submission

- Final deploy.
- Create cover image.
- Write 500-character description.
- Optional short YouTube demo.
- Submit production URL.

Workstreams (solo critical path, or AI subagents you orchestrate)

These are organizational buckets, not five people. Sequence them; optionally delegate a bucket to a subagent.

Workstream 1: Core Data And Scoring

Owns:

- `/api/github` serverless proxy (authenticated token + cache)
- GitHub API client (client side, calls the proxy)
- Sample data
- Activity normalization
- Energy score
- Evolution stage
- Builder class
- Error fallbacks

Deliverables:

- `GET /api/github?username=` proxy route (token + cache, maps statuses to UI states)
- `getGithubUser(username)` (client; calls the proxy)
- `normalizeGithubActivity(data)`
- `createPetProfile(activity)`
- Test fixtures for active, inactive, invalid, and sample users

Workstream 2: Frontend Experience

Owns:

- Landing screen
- Username input
- Hatch button
- Loading ritual
- Result layout
- Mobile responsiveness

Deliverables:

- Complete core user flow
- No-login path
- Sample username chips
- Empty/loading/error/result states

Workstream 3: Sprite And Visual System

Owns:

- Mascot art direction
- Pet variants
- Sprite component
- Mood/evolution display
- Animation states

Deliverables:

- Egg, Sprout, Builder, Evolved, Overclocked, Sleepy, Mystery variants
- Lightweight animations
- Asset fallback strategy

Workstream 4: Builder DNA And Copy

Owns:

- Builder classes
- Personality lines
- Mood labels
- Loading copy
- Submission description
- Demo script

Deliverables:

- Copy map keyed by mood/stage/language
- Builder DNA card content
- 500-character submission copy
- Judge demo script

Workstream 5: QA, Deploy, And Submission

Owns:

- Production deploy
- Mobile QA
- Browser checks
- API failure checks
- Submission asset checklist

Deliverables:

- Public URL
- Smoke test report
- Mobile screenshots
- Submission-ready checklist

Integration Contracts

PetProfile Contract

All UI should consume a single `PetProfile` object rather than raw GitHub API data.

Fields:

- username
- avatarUrl
- githubUrl
- level
- mood
- evolutionStage
- builderClass
- topLanguage
- energyScore
- activitySummary
- personalityLine
- spriteVariant
- outfitTrait

UI State Contract

States:

- idle
- loading
- success
- compare
- notFound
- rateLimited
- networkError

Feature Freeze Rule

After the core five features work, no new product features should be added unless:

- production deployment is already stable,
- mobile is already verified,
- invalid/rate-limited states work,
- and the feature improves the judging demo within 30 minutes.

Cut List If Time Gets Tight

Cut in this order:

1. Desk Mode
2. Share card export/download
3. Outfit overlays
4. Comparison highlight awards
5. Advanced language fetching

Never cut:

- GitHub hatch
- Pet state changes
- Builder DNA card
- Sample demo path
- Mobile-friendly layout

ShipGotchi Judging Demo And Submission Kit

Demo Goal

Make judges understand the app in 10 seconds and smile within 30 seconds.

Demo Script

1. Open ShipGotchi on mobile or desktop.
2. Say: "No login. Paste any public GitHub username."
3. Click a sample username.
4. Hatch animation plays.
5. Pet appears with mood, evolution, Builder DNA, and personality line.
6. Add another sample username to Compare Mode.
7. Show different pet states side-by-side.
8. End on the share/result card.

Three Features To List In Submission

1. GitHub username hatch: fetches public GitHub activity and creates a pet.
2. Activity-based pet evolution: mood, level, stage, outfit, and class change based on shipping energy.
3. Builder DNA and Compare Mode: polished identity card plus local side-by-side pet comparison.

Suggested Project Name

ShipGotchi

Suggested Tagline

Paste your GitHub. Meet the creature your commits created.

500-Character Submission Description

ShipGotchi turns any public GitHub username into a living builder pet. Paste a username, hatch a premium digital companion, and see its mood, evolution, level, outfit, and Builder DNA based on recent public GitHub activity. Compare friends locally, try sample builders with no login, and generate a shareable result card. It is a playful first step toward a persistent companion that grows as you ship.

Character count: 397

Tags

- github
- toy
- builder
- ai
- game
- no-login
- mobile
- devtools
- social
- dvc

Cover Image Direction

Composition:

- Large cute ShipGotchi mascot in center.
- GitHub username input behind or below.
- Builder DNA card peeking from side.

- Text: "Paste your GitHub. Hatch your ShipGotchi."

Mood:

- Bright premium toy.
- Soft glow.
- Mobile app screenshot feel.
- Avoid dark cyberpunk.

Judge Test Checklist

- Production URL loads publicly.
- No login is required.
- Input accepts a real GitHub username.
- Sample usernames work.
- Invalid username state works.
- Result shows pet state based on GitHub data.
- Result includes Builder DNA card.
- Compare Mode works.
- Mobile layout works.
- The app remains usable if GitHub API fails or rate-limits.

Recommended Sample Users

Use public, high-signal usernames for demo chips. Verify before launch.

- torvalds
- gaearon
- sindresorhus
- yyx990803

Add at least one bundled sample data profile so the demo works even if GitHub rate-limits.

Final Submission Safety

Before submitting:

- Confirm project URL is production (Vercel), not localhost.
- Confirm GITHUB_TOKEN is set in Vercel env and the `/api/github` proxy returns live data in production.
- Confirm the sample-data backstop still produces a full demo if GitHub/proxy fails.
- Confirm cover image is uploaded.
- Confirm description lists the working features.
- Confirm no required auth wall.
- Confirm URL will stay public through judging.
- Confirm mobile view is clean.

ShipGotchi Prompt Runbook

Date: 2026-05-22 Purpose: paste-by-paste prompts for building ShipGotchi from an empty project to a submitted build.

How To Use This Runbook

Use the prompts in order. Each prompt is written so it can be pasted into Codex or a specialist agent with minimal extra context. When running multiple agents, give each agent only the relevant prompt plus the planning docs it needs.

Recommended order:

1. Master setup prompt
2. Architecture and scaffold prompt
3. Core data/scoring agent prompt
4. Frontend experience agent prompt
5. Sprite/art direction agent prompt
6. Builder DNA/copy agent prompt
7. Integration prompt
8. QA and hardening prompt
9. Deployment prompt
10. Submission prompt

Do not add new features unless the five MVP features are stable.

Prompt 1: Master Setup

We are building ShipGotchi for the Dollar Vibe Club 24-Hour Open Build Sprint, Track 2: Vibe Gotchi.

Read the planning docs in this workspace before making implementation decisions:

- docs/00-shipgotchi-planning-index.md
- docs/01-product-brief.md
- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/04-technical-plan.md
- docs/05-execution-plan-agent-workstreams.md
- docs/06-judging-demo-submission-kit.md
- docs/07-prompt-runbook.md

Locked decisions (see planning index): solo full-stack builder, Next.js + TypeScript + Tailwind, Vercel hosting, a REQUIRED /api/github serverless proxy, and a layered (one

Goal:

Build a no-login, mobile-first consumer webapp where a visitor enters any public GitHub username and instantly hatches a premium digital pet whose mood, evolution, outfit,

MVP features:

1. GitHub Hatch
2. Pet Mood and Evolution
3. Premium Sprite Moment
4. Builder DNA Card
5. Compare / Party Mode

Rules:

- No login required.
- Webapp only.
- Public production URL.
- Mobile-friendly.
- Minimum 3 interactive features.
- Keep scope tight.
- Prioritize a stable judging demo over extra features.

First task:

Inspect the workspace, identify whether a project already exists, and recommend the fastest implementation path within the locked stack (Next.js + TS + Tailwind on Vercel).

Prompt 2: Scaffold And Architecture

Implement the ShipGotchi app scaffold.

Stack is locked: Next.js (App Router) + TypeScript + Tailwind CSS. We use Next specifically so the GitHub proxy lives in the same project as the app/api routes and deploys

Create a clean architecture:

- app shell / main page
- app/api/github route (serverless proxy – see Prompt 3)
- components for hatch form, loading ritual, pet display, Builder DNA card, compare mode, and error states
- a client service that fetches from /api/github (NOT the GitHub API directly)
- pure utility module for scoring and pet profile generation
- sample data fixtures for demo fallback
- asset folder for pet sprites

Constraints:

- No auth, no login.
- No database required.
- No runtime AI image generation.
- The UI must work with sample data even if the proxy/GitHub fails.
- Mobile-first.

After scaffolding:

1. Run install/build/lint and report results.
2. Deploy the empty scaffold to Vercel so a live production URL exists immediately, then report the URL.

Prompt 3: Core Data And Scoring Agent

You are the Core Data and Scoring agent for ShipGotchi.

Read:

- docs/02-mvp-specification.md
- docs/04-technical-plan.md
- docs/05-execution-plan-agent-workstreams.md

Your task:

Implement the GitHub proxy, data layer, and deterministic pet scoring system.

Deliver:

0. A serverless proxy at app/api/github (GET ?username=) that:
 - reads GITHUB_TOKEN from env (server-side only, never sent to the client)
 - fetches profile + recent public events + recent repos server-side
 - caches responses ~5 minutes to conserve GitHub quota
 - returns one normalized payload, or a typed error mapping 404 -> notFound, 403/rate-limit -> rateLimited, 5xx/network -> networkError

Why: unauthenticated GitHub is 60 req/hr per IP and will die during live judging; an authenticated proxy gives 5,000/hr plus caching.
1. A client GitHub service that fetches from /api/github (NOT api.github.com directly).
2. Normalization logic that produces a GitHubActivity object.
3. A createPetProfile(activity) function that returns:
 - username
 - avatarUrl
 - githubUrl
 - level
 - mood
 - evolutionStage
 - builderClass
 - topLanguage
 - energyScore
 - activitySummary
 - personalityLine
 - spriteVariant
 - outfitTrait
4. Sample fixture profiles for:
 - very active builder
 - normal builder
 - sleepy/inactive builder
 - old/legend account
 - invalid/rate-limited fallback
5. Unit tests or simple verification scripts for scoring outputs if the project has a test setup.

Important:

- Keep scoring simple, explainable, and deterministic.
- Never make low activity feel insulting.
- Handle rate limits, not found, network failure, and sparse data.
- Return a UI-friendly error state instead of throwing raw errors into components.

Acceptance:

Different sample users should produce visibly different moods, levels, classes, and sprite variants.

Prompt 4: Frontend Experience Agent

You are the Frontend Experience agent for ShipGotchi.

Read:

- docs/01-product-brief.md
- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/05-execution-plan-agent-workstreams.md

Your task:

Build the complete user flow from first screen to result.

Screens/states:

1. Landing / hatch state
2. Loading ritual
3. Result state
4. Error state
5. Compare / Party Mode state

Required UI:

- Brand: ShipGotchi
- Tagline: "Paste your GitHub. Meet the creature your commits created."
- Username input
- Hatch button
- Sample username chips
- Loading ritual with playful scan steps
- Pet result section
- Builder DNA card section
- Compare another builder action
- Up to 4 local comparison cards

Design direction:

- Cute premium toy.
- Light RPG stats.
- Soft builder-lab accents.
- Mobile-first.
- No dense dashboard feeling.

Important:

- The pet is the hero, not a small decoration.
- Keep text concise.
- Make the first successful hatch feel delightful within 10 seconds.
- Avoid feature explanation blocks on the page.

Acceptance:

A judge can open the site, click a sample username, see a pet result, and compare another username without needing instructions.

Prompt 5: Sprite And Art Direction Agent

You are the Sprite and Art Direction agent for ShipGotchi.

Read:

- docs/03-experience-visual-direction.md
- docs/02-mvp-specification.md

Your task:

Build the visual pet system as ONE base mascot with state-driven layers – not 7 independent images. Layers: body tint, expression, accessory, and motion. This guarantees th

Art direction:

Cute premium toy, soft digital collectible, expressive, friendly, mobile-friendly, screenshot-worthy. Add only light builder-lab/RPG flavor.

State targets (driven by the layers, all the same creature):

1. Egg
2. Sprout
3. Builder

- 4. Evolved
- 5. Overclocked
- 6. Sleepy
- 7. Mystery/error fallback

Optional overlays (accessory layer, by top language):

- TypeScript/JavaScript: star visor
- Python: soft wizard cap
- Rust: bronze armor accent
- Go: runner goggles
- CSS/HTML: paint scarf

Primary deliverable:

- An SVG/CSS sprite component that takes (spriteVariant, mood, outfitTrait) and renders the right layered combination, with light CSS/Framer Motion animation (idle bob, hat

Stretch deliverable (only after the 5 MVP features are stable):

- Swap in pre-generated AI renders per variant using runbook Prompt 6. Never make the demo depend on these.

Constraints:

- Do not depend on runtime AI image generation.
- Transparent or clean backgrounds.
- Variants must stay visually consistent as the same creature.
- Pet must be readable at mobile sizes.

Acceptance:

Each pet state must look distinct, premium, and emotionally readable.

Prompt 6: AI Image Generation Prompts

Use these prompts with an image-generation service if we create sprite assets externally.

Global Style Prompt

Create a cute premium digital toy mascot for a webapp called ShipGotchi. The mascot is a small builder companion born from GitHub commits. Style: soft 3D vinyl toy, collec

Egg Variant

Create the Egg form of the ShipGotchi mascot. A cute premium toy egg with tiny glowing code-symbol freckles, soft rounded shape, curious expression hinted through a small

Sprout Variant

Create the Sprout form of the ShipGotchi mascot. Same character as the egg, now a tiny newly hatched builder creature with small rounded body, bright eyes, tiny leaf-like

Builder Variant

Create the Builder form of the ShipGotchi mascot. Same character, now confident and active, wearing tiny soft builder goggles and a small utility scarf, holding a glowing

Evolved Variant

Create the Evolved form of the ShipGotchi mascot. Same character, more polished and heroic, larger expressive eyes, subtle badge on chest, tiny floating code sparks around

Overclocked Variant

Create the Overclocked form of the ShipGotchi mascot. Same character, excited and powered up, tiny glowing goggles, energetic pose, harmless bright commit sparks, premium

Sleepy Variant

Create the Sleepy form of the ShipGotchi mascot. Same character, cozy and napping, soft blanket or tiny hoodie, dim commit spark beside it, sweet and lovable, not sad or p

Mystery Variant

Create the Mystery fallback form of the ShipGotchi mascot. Same character partly hidden in a soft translucent mystery bubble, curious eyes visible, gentle question mark-sh

Prompt 7: Builder DNA And Copy Agent

You are the Builder DNA and Copy agent for ShipGotchi.

Read:

- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/06-judging-demo-submission-kit.md

Your task:

Create the personality/copy system that makes the app feel alive.

Deliver:

1. Builder class names mapped to common activity/language patterns.
2. Mood labels and descriptions.
3. Evolution descriptions.
4. Loading ritual copy.
5. Personality lines for each mood/class combination.
6. Error copy for invalid username, rate limit, network failure, and sparse data.
7. Final submission description under 500 characters.

Tone:

Warm, playful, premium, builder-native, never cruel.

Avoid:

- Long explanations
- Mean roasts
- Overused developer memes
- Copy that requires insider knowledge to understand

Acceptance:

The copy should make the result feel personal and screenshot-worthy while staying clear for judges.

Prompt 8: Integration Prompt

Integrate the ShipGotchi workstreams into one polished product.

Read all docs in /docs, especially:

- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/04-technical-plan.md
- docs/05-execution-plan-agent-workstreams.md

Tasks:

1. Wire the hatch form to the GitHub data service.
2. Convert GitHub data into a PetProfile.
3. Render the correct pet sprite variant from PetProfile.spriteVariant.
4. Render Builder DNA from the PetProfile contract.
5. Add local Compare Mode for up to 4 users.
6. Add sample username chips and bundled sample data fallback.
7. Ensure loading, success, not found, rate-limited, and network error states all look intentional.
8. Make the app responsive and polished on mobile.

Do not add new major features.

Acceptance:

The full user flow works from a clean page load:

landing -> hatch -> loading -> pet result -> compare mode -> share/result card.

Prompt 9: QA And Hardening Agent

You are the QA and Hardening agent for ShipGotchi.

Read:

- docs/02-mvp-specification.md
- docs/04-technical-plan.md
- docs/06-judging-demo-submission-kit.md

Your task:

Break the app before judges do.

Test:

1. Valid active GitHub usernames.
2. Valid inactive GitHub usernames.
3. Invalid usernames.
4. Empty input.
5. GitHub API rate-limit behavior.
6. Network failure if possible.
7. Mobile viewport.
8. Desktop viewport.
9. Compare Mode with 2, 3, and 4 users.
10. Fresh production deployment URL.

Required fixes:

- No raw errors visible to users.
- No login required.
- No layout overlap on mobile.
- No broken sprite states.
- Sample data path must work without GitHub.

Report:

- What passed.
- What failed.
- What was fixed.
- Remaining risks.

Prompt 10: Deployment Agent

Deploy ShipGotchi to a public production URL.

Read:

- docs/06-judging-demo-submission-kit.md

Tasks:

1. Build the app locally.
2. Fix build errors.
3. Deploy to Vercel (Git-connected, auto-deploy on push).
4. Confirm GITHUB_TOKEN is set in Vercel env and the /api/github proxy returns live data in production.
5. Verify the production URL in a browser.
6. Test the core judge path on production:
 - open URL
 - click sample username
 - hatch result
 - compare another user
 - view Builder DNA
7. Confirm no login is required.
8. Confirm mobile layout is usable.
9. Hammer the proxy: repeat fetches + a 4-way compare and confirm caching keeps it from rate-limiting; confirm sample-data backstop still works if the proxy is forced to fa

Deliver:

- Production URL
- Smoke test summary
- Any limitations judges should know, if unavoidable

Prompt 11: Submission Assets Agent

Prepare ShipGotchi submission assets for Dollar Vibe Club.

Read:

- docs/06-judging-demo-submission-kit.md

Deliver:

1. Final project name.
2. Final tagline.
3. Final under-500-character submission description.
4. 10 tags.
5. Cover image concept or actual cover image.
6. Optional 60-90 second demo video script.
7. Three working features to list clearly.

Submission details:

Project name: ShipGotchi

Tagline: Paste your GitHub. Meet the creature your commits created.

Working features:

1. GitHub username hatch.
2. Activity-based pet evolution.
3. Builder DNA and Compare Mode.

Make the submission sound polished, launchable, and easy to judge.

Prompt 12: Final Judge Demo Rehearsal

Run a final judge demo rehearsal for ShipGotchi.

Use the production URL.

Demo path:

1. Open the app.
2. Click a sample username.
3. Wait for hatch.
4. Explain the pet mood, evolution, level, and Builder DNA.
5. Add another username to Compare Mode.
6. Show side-by-side difference.
7. Show share/result card.

Time target:

The core value should be obvious within 30 seconds.

Check:

- Is the first screen instantly understandable?
- Does the pet feel like the hero?
- Are the five MVP features visible?
- Does the app recover gracefully from bad input?
- Is mobile clean?
- Would a stream audience understand it without explanation?

Deliver:

A final go/no-go recommendation and a short list of only must-fix issues.

Prompt 13: Emergency Scope Cut

Use this prompt if time is running out.

We are close to deadline. Cut ShipGotchi down to the strongest shippable judging demo.

Protect these features:

1. GitHub hatch or sample-data hatch.
2. Pet mood/evolution visual.
3. Builder DNA card.
4. Mobile-friendly production URL.
5. No-login judging path.

Cut in this order:

1. Desk Mode.
2. Share card export/download.
3. Outfit overlays.
4. Advanced comparison awards.
5. Advanced language fetching.

Task:

Identify unstable or unfinished features, remove or hide them, and make the remaining path feel intentional and polished.

Acceptance:

A judge can still complete a delightful demo in under 60 seconds.

Prompt 14: Post-Sprint Vision Pitch

Use this for the submission, demo narration, or follow-up if ShipGotchi places.

Write a concise post-sprint vision pitch for ShipGotchi.

Positioning:

ShipGotchi starts as a no-login web toy that turns public GitHub activity into a living builder pet. After the sprint, it can become a persistent companion that lives beside

Future expansion ideas:

- Desktop/web companion mode
- Browser extension
- Team pets
- Streak celebrations
- Seasonal evolution forms
- Shareable launch cards
- DVC builder community leaderboard

Keep it concise, exciting, and realistic. Emphasize that the sprint MVP already proves the core loop: public activity -> emotional pet -> shareable identity.

Completion Checklist

Before submitting, confirm:

- The app is deployed to a public Vercel URL.
- GITHUB_TOKEN is set in Vercel env and the /api/github proxy returns live data in production.
- Caching confirmed: repeated fetches + a 4-way compare do not rate-limit.
- Sample-data backstop works even if the proxy is forced to fail.
- No login is required.
- Sample usernames work.
- At least one real GitHub username works.
- Invalid username state works.
- Pet state visibly changes across users.
- Builder DNA card is readable on mobile.
- Compare Mode works.
- Cover image is ready.
- Submission copy is under 500 characters.
- The production URL will stay online through judging.

ShipGotchi Sprite Asset Prompts (Premium Render Track)

Purpose: generate best-in-class mascot art that can be swapped in to replace (or sit on top of) the built-in SVG sprite. The SVG already ships a working, on-brand creature; these renders are the polish upgrade. **Do not let asset generation block the build** — drop PNGs in when they're ready.

The non-negotiable: ONE creature, seven states

The single biggest failure mode is seven renders that look like seven different animals. Everything below is engineered to prevent that. Generate the **reference sheet first**, lock the character, then derive every state from it.

Character Bible (paste this verbatim into every prompt)

CHARACTER: "ShipGotchi" — a small, round designer-toy creature; a loyal builder companion hatched from code.
BODY: chibi proportions, oversized head on a soft egg/bean-shaped body (~1.2:1 tall), short stubby arms, two small rounded feet. Smooth premium vinyl finish with a soft sat
FACE: two oversized glossy black eyes with twin white catchlights, small rosy round cheeks, a tiny simple mouth. Warm, friendly, a little mischievous.
SIGNATURE: a two-prong antenna shaped like an angle-bracket pair "< >" curling up from the top of the head. This antenna **MUST** appear on every form (except the Egg).
DEFAULT COLORWAY: warm coral-orange body with a soft cream belly highlight (state colorways override this where noted).
STYLE: high-end collectible designer toy (Pop Mart / Sonny Angel energy), soft 3D render, studio softbox lighting, gentle rim light, subtle ambient occlusion, clean rounded
MOOD OF BRAND: cute, premium, tactile, screenshot-worthy. Bright and friendly.

Negative / avoid (append to every prompt)

AVOID: dark cyberpunk, neon grunge, horror, sharp teeth, realistic animal, human face, text, watermark, logos, busy background, clutter, harsh shadows, low-res, flat clip-

Output spec (every asset)

- 1:1 square, **transparent background** (or solid #FFF6EC then remove), centered, with ~12% padding on all sides so scale is consistent across variants.
- Same camera: straight-on, slight 5° high angle, eye-level-ish, identical zoom/scale between variants.
- 1024×1024 minimum (2048 preferred for crisp retina). Export PNG.
- Keep the contact shadow inside the frame; no cropping of antenna or feet.

Step 1 — Reference sheet (generate this FIRST)

[CHARACTER BIBLE]
Create a clean character reference of this single creature in its default coral-orange colorway: front view, neutral happy expression, standing, full body, centered, soft s
Pick the best result. That image is your **anchor**. Use it as the reference for every variant below.

Step 2 — Lock consistency by model

- Midjourney**: generate the anchor, then for each variant use `--cref <anchor_url> --cw 100` (character reference, full weight) and a fixed `--seed`. Add `--sref <anchor_url>` to also lock the rendering style.
- Nano Banana / Gemini 2.5 Flash Image**: upload the anchor and prompt "Using the SAME character shown, render it as ... keep identical body shape, colors, eyes, and antenna." Iterate by re-feeding the anchor each time.
- GPT Image (ChatGPT)**: upload the anchor and ask for an edit/restyle into each state, repeating "same character, same proportions and colors."
- Higgsfield / others**: use their image-reference / character-lock feature with the anchor; otherwise paste the full Character Bible and a strict "same creature" instruction every time.

Always render the **Builder** form second (it's the most "default") to confirm the anchor reproduces before doing the rest.

Step 3 — Per-variant prompts

Each variant = [CHARACTER BIBLE] + the block below + [OUTPUT SPEC] + [AVOID] and the model's reference-lock from Step 2.

Egg (/public/sprites/egg.png)

STATE — EGG: the creature is still inside its egg, not yet hatched. A smooth coral-orange speckled egg with a soft glossy sheen and a small zig-zag crack across the middle

Sprout (/public/sprites/sprout.png)

STATE – SPROUT: a tiny, newly hatched version of the creature – smaller, rounder, a little wobbly. Bright wide eyes, eager expression. The "< >" antenna is small and fresh

Builder (/public/sprites/builder.png)

STATE – BUILDER: the confident everyday form. Standing proud, content half-smile, bright eyes. Wearing tiny soft builder goggles pushed up on the forehead and a small util

Evolved (/public/sprites/evolved.png)

STATE – EVOLVED: a more heroic, polished form. Slightly taller and sleeker, larger expressive eyes, a small glowing achievement badge on its chest. A few tiny code-sparks

Overclocked (/public/sprites/overclocked.png)

STATE – OVERCLOCKED: powered-up and joyful, mid-celebration. Big open grin, sparkling eyes, energetic pose (one arm raised). A bright warm aura/halo glows behind it; harmI

Sleepy (/public/sprites/sleepy.png)

STATE – SLEEPY: cozy and napping, sweet not sad. Eyes closed in soft happy curves, tiny content smile, wrapped in a small blanket or wearing a little nightcap. A dim commi

Mystery (/public/sprites/mystery.png)

STATE – MYSTERY (fallback): the creature half-hidden inside a soft translucent iridescent bubble, only its curious glossy eyes clearly visible. A gentle question-mark-shap

Optional — language outfit overlays

Best baked into a couple of hero variants rather than as separate layers. If you want standalone transparent accessories (to composite over any body), render each on its own with: "single accessory only, transparent background, no character, product shot, soft-3D toy style."

- TypeScript / JavaScript → **Star Visor** (sporty visor band with a small gold star)
- Python → **Wizard Cap** (soft pointed cap with a star)
- Rust → **Bronze Armor** (small bronze shoulder pauldrons)
- Go → **Runner Goggles** (mint racing goggles)
- CSS / HTML → **Paint Scarf** (sky-blue flowing scarf)
- Java → **Coffee Charm** (tiny coffee-cup charm)

Step 4 — Post-process + integrate

1. Remove background to clean alpha (if not already transparent); trim, then re-center each on an identical square canvas so all 7 share scale + baseline.
2. Export to `public/sprites/{egg,sprout,builder,evolved,overclocked,sleepy,mystery}.png` .
3. Wiring: extend `ShipGotchiSprite` with an optional image layer — when a PNG exists for the variant, render `` (with the existing CSS `bob/animation` on the wrapper) and fall back to the SVG creature if the image 404s or hasn't been added. This keeps the demo bulletproof: no asset = still a working pet.
4. Re-check the Builder DNA card and Compare cards at small sizes — the PNGs must read clearly at 96px.